

Implementation of Hybrid RAG BM25 Keyword Search + Dense Semantic Search + Reciprocal Rank Fusion E-Commerce Analytics System — Olist Brazilian Dataset RAGAS & DeepEval Evaluation Framework | May 2026

Abstract

This document describes the end-to-end implementation of a Hybrid RAG system built on the Olist Brazilian e-commerce dataset. The system combines two complementary retrieval paradigms: **BM25 keyword search** (exact term matching via TF-IDF-weighted bag-of-words) and **dense semantic search** (cosine similarity over all-MiniLM-L6-v2 embeddings in ChromaDB). The two ranked lists are merged using **Reciprocal Rank Fusion (RRF, $k=60$)** to produce a unified top-5 candidate set, which is passed to *llama-3.3-70b-versatile* via the Groq API for answer generation. Hybrid RAG achieves the highest evaluation scores among all implemented RAG variants, with Context Precision 0.545 and Context Recall 0.564.

1. System Overview

Hybrid RAG addresses two distinct failure modes in single-method retrieval. Dense semantic search excels at conceptual similarity ('logistics efficiency' matching 'delivery performance') but fails at exact token matching (seller UUIDs, specific numeric values, date strings). BM25 keyword search excels at exact term overlap but cannot generalise across synonyms or paraphrases. By running both methods in parallel and fusing their ranked lists, Hybrid RAG captures documents that either method would miss independently.

1.1 Architecture Diagram

Layer	Component	Technology	Role
Data	Raw Olist CSV Files	9 CSV files / 99K-1M rows each	Source data for KB construction
Processing	ETL Pipeline (5 steps)	Python / Pandas	Join, enrich, aggregate to KB docs
Knowledge	kb_all_documents.json	13,225 JSON documents	Structured KB across 6 document types
Index A	ChromaDB Vector Store	all-MiniLM-L6-v2 (384-dim, cosine)	Dense semantic index
Index B	BM25 Pickle Index	rank_bm25.BM25Okapi, tokenized corpus	Sparse keyword (exact-term) index
Retrieval A	Semantic search	ChromaDB query API, top-10	Conceptual similarity retrieval
Retrieval B	BM25 keyword search	bm25.get_scores(tokenize(query))	Exact-term overlap retrieval
Fusion	RRF	k=60, final top-5	Merge both ranked lists
Generation	hybrid_rag/generator.py	llama-3.3-70b-versatile / Groq API	Context-grounded answer synthesis
Evaluation	evaluation/ scripts	RAGAS + DeepEval + golden dataset	Reference-based 0-LLM-judge metrics

2. Data Preparation Pipeline

Hybrid RAG requires two indexes: the ChromaDB vector store (shared with Naive, HyDE, Reranking, and Multi-Query RAG) and a BM25 pickle index unique to this variant. Both are built from the same 13,225-document knowledge base in a single-pass ingestion via `build_all()`.

2.1 ETL Steps

Step	Script	Output	Description
1	step1_load_raw_data.py	9 validated DataFrames	Load CSVs, validate schema, report nulls
2	step2_join_datasets.py	master_joined.csv	Star-schema join on order_id / customer_id / product_id
3	step3_enrich_master.py	master_enriched.csv	Add delivery_days, late_flag, review_score_category
4	step4_build_knowledge_base.py	kb_all_documents.json	Aggregate to 6 document-type layers (13,225 docs)
5	step5_build_golden_dataset.py	golden_dataset.csv	Generate 100 Q&A pairs via Gemini Flash LLM

2.2 Knowledge Base Document Types

Layer	ID Pattern	Count	Fields
Order	order_<uuid>	~99K	Order ID, payment, delivery dates, review, freight
Seller	seller_<id>	~3K	Seller city/state, avg review, total revenue, late rate

Layer	ID Pattern	Count	Fields
Product Category	category_<name>	~73	Category name, total orders, avg review, revenue, late rate
Delivery Status	delivery_status_<type>	3	early / on_time / late — aggregate stats per group
Customer State	state_<abbr>	~27	State-level orders, satisfaction, logistics metrics
Temporal	month_<YYYY_MM>	~25	Month-level: total orders, avg payment, top categories

3. Dual Index Ingestion

Hybrid RAG maintains two indexes over the same document corpus. The `build_all()` function loads the knowledge base once and passes it to both `build_chroma()` and `build_bm25()`, avoiding double disk I/O during the single-pass ingestion.

3.1 ChromaDB Semantic Index

Parameter	Value	Notes
Embedding model	all-MiniLM-L6-v2	384-dim; shared collection with all other RAG types
Distance metric	cosine	hnsw:space = cosine in collection metadata
Batch size	500 documents	Memory-efficient batch ingestion
Collection name	ecommerce_kb	Reused across all RAG types — no re-embedding needed
Persistence	chroma_db/	PersistentClient; survives Python restarts

3.2 BM25 Keyword Index

The BM25 index is built using the `rank_bm25` library's BM25Okapi implementation. Each document is tokenised with `tokenize()` (lowercase, strip punctuation, split on whitespace) and the full 13,225-document corpus is fitted into a single BM25Okapi object. The fitted model and the original document list are pickled to disk for fast reuse across sessions.

Parameter	Value	Notes
Library	rank_bm25.BM25Okapi	BM25 with Okapi TF saturation
Tokeniser	tokenize() from utils.py	Lowercase, strip punctuation, whitespace split
Corpus size	13,225 documents	All KB documents — no filtering
Storage	dataset/bm25_index/bm25_index.pkl	Pickle: {'bm25': BM25Okapi, 'docs': list[dict]}
Zero-score docs	Excluded from results	Docs with BM25 score=0 add no retrieval signal
Singleton cache	Module-level _bm25_cache	Loaded from disk once per session; in-memory after

3.3 Tokenisation Function

```
tokenize(text: str) -> list[str] (utils.py)

import re
return re.sub(r'[\^\w\s]', ' ', text.lower()).split()

Example:
'health_beauty category: 9,672 orders.'
-> ['health_beauty', 'category', '9', '672', 'orders']

Used by: build_bm25() (corpus tokenisation) and
_keyword_search() (query tokenisation)
Consistent tokenisation ensures vocabulary alignment between
index build time and query time.
```

4. Dual Retrieval

For each query the retriever runs BM25 keyword search and ChromaDB semantic search in parallel (sequential in the current implementation), then fuses the two ranked lists via RRF. The public `retrieve()` function returns all three lists — fused, keyword, and semantic — enabling per-query analysis of each method's contribution.

4.1 BM25 Keyword Search

```
_keyword_search(query, top_k=10) -> list[dict]:

1. bm25, docs = get_bm25_index() # cached singleton
2. tokens = tokenize(query) # same tokeniser as build
3. scores = bm25.get_scores(tokens) # BM25 score per document
4. Filter: keep only docs where score > 0 # exclude zero-overlap docs
5. Sort descending by score
6. Return top_k dicts: {id, text, metadata, bm25_score}

KEYWORD_TOP_K = 10 (config.py)
```

4.2 Semantic Search

```
_semantic_search(query, top_k=10) -> list[dict]:

1. col = get_chroma_collection() # cached singleton
2. res = col.query(query_texts=[query],
n_results=top_k) # embed + cosine search
3. Return top_k dicts: {id, text, metadata, distance}

SEMANTIC_TOP_K = 10 (config.py)
```

4.3 Why Exclude Zero-Score BM25 Documents?

A BM25 score of exactly 0.0 means the query and the document share no token overlap whatsoever. Including such documents in the RRF fusion would assign them a minimum rank contribution of $1/(60+\text{top_k}) = 1/70 \approx 0.014$, which could promote them above genuinely relevant semantic results that scored low in the semantic ranking. Filtering zero-score BM25 results ensures that only documents with actual keyword evidence contribute to the BM25 side of the RRF equation.

5. Reciprocal Rank Fusion (RRF)

RRF merges the BM25 and semantic ranked lists into a single fused ranking. A document appearing in both lists accumulates scores from both — making cross-method consensus a strong promotion signal. Documents that score well in both keyword and semantic search are reliably the most relevant.

5.1 RRF Algorithm

```
_rrf_fusion(keyword_results, semantic_results, k=60, final_top_k=5):

    rrf_scores = {} # doc_id -> cumulative score
    doc_store = {} # doc_id -> doc dict (last seen copy)

    for rank, doc in enumerate(keyword_results, start=1):
        rrf_scores[doc['id']] = rrf_scores.get(doc['id'], 0.0) + 1.0/(k+rank)
        doc_store[doc['id']] = doc

    for rank, doc in enumerate(semantic_results, start=1):
        rrf_scores[doc['id']] = rrf_scores.get(doc['id'], 0.0) + 1.0/(k+rank)
        doc_store[doc['id']] = doc

    top_ids = sorted(rrf_scores, key=lambda d: rrf_scores[d],
                    reverse=True)[:final_top_k]
    return [{**doc_store[id], 'rrf_score': rrf_scores[id]} for id in top_ids]
```

5.2 RRF Score Example

Document	BM25 rank	Semantic rank	RRF score	Why
category_health_beauty	1	1	0.0328	Appears in both lists at rank 1
seller_uuid_3b4c...	2	N/A	0.0161	BM25 only — exact UUID match
state_SP	N/A	2	0.0161	Semantic only — conceptual match
month_2018_08	8	5	0.0147	Both lists, lower ranks
order_uuid_a1b2...	N/A	N/A	0.0000	Neither — not in top-10 of either

5.3 Configuration

Parameter	Value	Notes
RRF_K	60	Standard constant; reduces over-weighting of top-rank docs
SEMANTIC_TOP_K	10	Semantic candidates before fusion
KEYWORD_TOP_K	10	BM25 candidates before fusion (zero-score excluded)
FINAL_TOP_K	5	Fused docs passed to LLM

6. Answer Generation Module

The generator receives the original query and the top-5 RRF-fused documents and calls **llama-3.3-70b-versatile** via Groq at temperature=0.1. The prompt template is identical to Naive RAG — only the context documents differ (they now reflect fused BM25+semantic ranking).

Parameter	Value	Rationale
Model	llama-3.3-70b-versatile	State-of-the-art open model via Groq
Temperature	0.1	Low randomness for factual e-commerce answers
Max tokens	512	Sufficient for analytical answers
Groq calls/q	1	Generation only — no query expansion needed

7. End-to-End Pipeline

7.1 Execution Flow

Step	Function	Input -> Output
0 — Init	build_all()	kb_all_documents.json -> ChromaDB + BM25 pickle
1 — BM25	_keyword_search(query, top_k=10)	query -> [10 docs with bm25_score]
2 — Semantic	_semantic_search(query, top_k=10)	query -> [10 docs with distance]
3 — Fuse	_rrf_fusion(kw, sem, k=60, top_k=5)	2 ranked lists -> 5 fused docs with rrf_score
4 — Generate	generate(query, fused_docs)	query + 5 docs -> answer string
5 — Return	run_hybrid_rag(query)	query -> {query, answer, retrieved_docs, keyword_docs, semantic_docs}

7.2 Pipeline Return Schema

```
run_hybrid_rag(query) returns:
{
  'query' : str — original question
  'answer' : str — LLM-generated answer
  'retrieved_docs': list[dict] — top-5 RRF-fused docs
  each has: id, text, metadata, rrf_score
  + bm25_score (if from BM25 list)
  + distance (if from semantic list)
  'keyword_docs' : list[dict] — raw BM25 top-10 (for analysis)
  'semantic_docs' : list[dict] — raw semantic top-10 (for analysis)
}
```

7.3 Interactive Entry Point

```
$ python run_hybrid_rag.py
```

Question: What is the total revenue for seller 3b4c29...?

Retrieved Documents (RRF-fused):
[1] rrf=0.033 seller_3b4c29... (in BM25 rank 1 + semantic rank 3)
[2] rrf=0.016 category_electronics
...

Answer: The total revenue for seller 3b4c29... is R\$42,381.00.

\$ python run_hybrid_rag.py --build-index # rebuild both indexes

8. Key Dependencies

Library	Version	Purpose
chromadb	>=0.5.0	Dense semantic vector store
sentence-transformers	>=3.0.0	all-MiniLM-L6-v2 embedding model
rank_bm25	>=0.2.0	BM25Okapi keyword index
groq	>=0.11.0	Groq API client for LLM inference
pandas / numpy	>=2.0	Data manipulation
ragas	>=0.2.0	RAGAS evaluation framework
deepeval	>=1.0.0	DeepEval evaluation framework
scikit-learn	latest	TF-IDF and cosine similarity for eval metrics
openpyxl	>=3.1.0	Excel export for evaluation results
reportlab	>=4.0.0	PDF report generation
python-dotenv	>=1.0.0	Environment variable management

9. Configuration Reference

Config Key	Default	Description
GROQ_API_KEYS	env var	Comma-separated list of Groq API keys
EMBEDDING_MODEL	all-MiniLM-L6-v2	Dense embedding model for ChromaDB
GROQ_MODEL	llama-3.3-70b-versatile	Groq LLM model identifier
COLLECTION_NAME	ecommerce_kb	ChromaDB collection name (shared)
CHROMA_DB_PATH	chroma_db/	ChromaDB persistence directory
BM25_INDEX_PATH	dataset/bm25_index/bm25_index.pkl	BM25 pickle file path
SEMANTIC_TOP_K	10	Semantic candidates before RRF
KEYWORD_TOP_K	10	BM25 candidates before RRF

Config Key	Default	Description
FINAL_TOP_K	5	Fused docs after RRF sent to LLM
RRF_K	60	RRF smoothing constant

10. Summary

Hybrid RAG combines the strengths of two fundamentally different retrieval paradigms. BM25 excels at exact-term matching — critical for seller UUIDs, order IDs, specific category names, and numeric values embedded in document text. Dense semantic search excels at conceptual similarity — matching analytical questions about 'logistics efficiency' to documents about 'delivery performance' without exact vocabulary overlap. Reciprocal Rank Fusion provides a theoretically sound, parameter-light mechanism to combine both signals, with documents appearing in both lists receiving a strong promotion bonus. The result is the best-performing RAG variant in this project, achieving Context Precision 0.545 and Context Recall 0.564.